

Week08 Redis 场景案例

1. 项目说明

作业实现了两个常见业务场景：

1. 短信验证码功能
2. 点赞功能

当前项目中的主要功能包如下：

- com.hxc.springbootweek08.sms
 - com.hxc.springbootweek08.like
-

2. 场景一：短信验证码

2.1 功能描述

短信验证码功能用于在登录、注册、找回密码等场景中验证用户手机号是否有效。系统接收到手机号后，生成一个短期有效的验证码，模拟发送短信，并将验证码缓存到 Redis。用户提交验证码后，再从 Redis 中读取并完成校验。

2.2 设计思路

短信验证码场景适合使用 Redis，主要原因如下：

- 验证码是典型的短期数据，适合设置过期时间
- 读写频率高，但数据结构简单
- 不需要持久化到数据库
- Redis 支持高效的过期控制和计数操作

该功能主要包含以下几个设计点：

1. 验证码缓存 使用 sms:code:{phone} 保存验证码，并设置 5 分钟有效期。
2. 发送冷却时间 使用 sms:cooldown:{phone} 保存发送冷却标记，限制同一手机号 60 秒内不能重复发送。
3. 每日发送次数限制 使用 sms:count:{yyyyMMdd}:{phone} 记录每日发送次数，防止短信接口被滥用。
4. 验证成功后立即删除 验证码校验通过后，立即从 Redis 中删除，避免重复使用。

2.3 实现方法

短信验证码功能相关代码统一放在 sms 包下：

- SmsVerifyCodeController
- SmsVerifyCodeService
- SmsVerifyCodeServiceImpl
- SmsSender
- MockSmsSender
- SmsRedisKey
- sms/dto

其中：

- SmsVerifyCodeController 负责提供接口
- SmsVerifyCodeServiceImpl 负责验证码生成、频控、校验逻辑
- MockSmsSender 负责模拟短信发送，当前阶段只打印日志
- SmsRedisKey 统一管理 Redis Key 命名规则

2.4 接口说明

发送验证码

- 路径：POST /api/sms/send
- 请求体：

```
{
  "phone": "13900003333"
}
```

- 返回示例：

```
{
  "message": "验证码发送成功",
  "expireSeconds": 300
}
```

校验验证码

- 路径: POST /api/sms/verify
- 请求体:

```
{
  "phone": "13900003333",
  "code": "123456"
}
```

- 返回示例:

```
{
  "success": true,
  "message": "验证码校验通过"
}
```

2.5 Redis 数据示例

```
sms:code:13900003333
sms:cooldown:13900003333
sms:count:20260423:13900003333
```

2.6 适用场景

- 手机号登录
- 注册校验
- 找回密码
- 手机号绑定

3. 场景二：点赞功能

3.1 功能描述

点赞功能用于记录某个用户是否对某个业务对象点过赞，并支持取消点赞和查询当前点赞状态。这里的业务对象可以是：

- 帖子
- 评论
- 视频
- 商品

3.2 设计思路

点赞功能非常适合使用 Redis 的 Set 类型实现，因为 Set 天然支持去重。

使用 Redis Set 的好处：

- 同一个用户重复点赞不会重复计数
- 可以快速判断某用户是否已点赞
- 可以直接获取点赞总数
- 实现简单，性能较好

设计上，一个业务对象对应一个点赞集合：

- Key: like:set:{bizType}:{bizId}
- Value: 用户 ID 集合

例如：

```
like:set:post:1001
```

表示业务类型为 post、业务 ID 为 1001 的对象，其点赞用户列表保存在这个集合里。

3.3 实现方法

点赞功能相关代码统一放在 like 包下：

- LikeController
- LikeService
- LikeServiceImpl
- LikeRedisKey
- LikeExceptionHandler
- like/dto

其中：

- LikeController 负责对外提供点赞、取消点赞、查询状态接口
- LikeServiceImpl 负责 Redis 中点赞集合的增删查逻辑
- LikeRedisKey 统一管理点赞功能的 Redis Key
- like/dto 中存放请求对象和视图对象

3.4 接口说明

点赞

- 路径: POST /api/likes
- 请求体:

```
{
  "bizType": "post",
  "bizId": "1001",
  "userId": "u10001"
}
```

- 返回示例:

```
{
  "success": true,
  "message": "操作成功",
  "data": {
    "success": true,
    "message": "点赞成功",
    "liked": true,
    "likeCount": 1
  }
}
```

取消点赞

- 路径: POST /api/likes/cancel
- 请求体:

```
{
  "bizType": "post",
  "bizId": "1001",
  "userId": "u10001"
}
```

查询点赞状态

- 路径: GET /api/likes/status
- 请求参数:

```
bizType=post
bizId=1001
userId=u10001
```

- 返回示例:

```
{
  "success": true,
  "message": "查询成功",
  "data": {
    "bizType": "post",
    "bizId": "1001",
    "userId": "u10001",

```

```
        "liked": true,  
        "likeCount": 1  
    }  
}
```

3.5 Redis 数据示例

like:set:post:1001

集合中的成员示例：

```
u10001  
u10002  
u10003
```

3.6 适用场景

- 帖子点赞
- 评论点赞
- 视频点赞
- 商品收藏或喜欢

4. 项目中的 Redis 使用总结

本项目通过两个案例，演示了 Redis 在业务开发中的两种典型使用方式：

1. 使用字符串类型保存短期状态 适合验证码、令牌、一次性校验值等场景。
2. 使用集合类型保存用户行为 适合点赞、收藏、关注、投票等去重型业务场景。

可以看到，Redis 不只是缓存数据库查询结果，更适合承载这类高频、轻量、状态性强的业务数据。

5. 测试说明

当前项目中已经包含相关测试：

- StringRedisTemplateTest
- RedisServiceTest
- SmsCodeServiceTest
- LikeServiceTest

执行命令：

```
./mvnw test
```